SoundStream: An End-to-End Neural Audio Codec

Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi, *Member, IEEE*

***Abstract*—We present *SoundStream*, a novel neural audio codec that can efficiently compress speech, music and general audio at bitrates normally targeted by speech-tailored codecs. *SoundStream* relies on a model architecture composed by a fully convolutional encoder/decoder network and a residual vector quantizer, which are trained jointly end-to-end. Training leverages recent advances in text-to-speech and speech enhancement, which combine adversarial and reconstruction losses to allow the generation of high-quality audio content from quantized embeddings. By training with structured dropout applied to quantizer layers, a single model can operate across variable bitrates from 3 kbps to 18 kbps, with a negligible quality loss when compared with models trained at fixed bitrates. In addition, the model is amenable to a low latency implementation, which supports streamable inference and runs in real time on a smartphone CPU. In subjective evaluations using audio at 24 kHz sampling rate, *SoundStream* at 3 kbps outperforms Opus at 12 kbps and approaches EVS at 9.6 kbps. Moreover, we are able to perform joint compression and enhancement either at the encoder or at the decoder side with no additional latency, which we demonstrate through background noise suppression for speech.**

***Index Terms*—Audio compression, codecs, convolution, neural networks, speech enhancement.**

I. Introduction

AUDIO codecs can be partitioned into two broad categories: waveform codecs and parametric codecs. Waveform codecs aim at producing at the decoder side a faithful reconstruction of the input audio samples. Often these codecs rely on transform coding techniques: a (usually invertible) transform is used to map an input time-domain waveform to the time-frequency domain. Then, transform coefficients are quantized and entropy coded. At the decoder side the transform is inverted to reconstruct a time-domain waveform. Many codecs today combine transform coding with linear predictive coding in the time-domain, especially at medium bitrates and/or narrower signal bandwidths. The bit allocation at the encoder is driven by a perceptual model, which determines the quantization process. Generally, waveform codecs make little or no assumptions about the type of audio content and can thus operate on general audio. As a consequence of this, they produce very high-quality audio at medium-to-high bitrates, but they tend to introduce coding artifacts when operating at low bitrates. Parametric codecs [1] take a different approach, by making specific assumptions about the source audio to be encoded (in most cases, speech) and introducing strong priors in the form of a parametric model that describes the audio synthesis process. The encoder estimates the parameters of the model, which are then quantized. The decoder generates a time-domain waveform using a synthesis model driven by quantized parameters. Unlike waveform codecs, the goal is not to obtain a faithful reconstruction on a sample-by-sample basis, but rather to generate audio that is perceptually similar to the original.

Traditional waveform and parametric codecs rely on signal processing pipelines and carefully engineered design choices, which exploit in-domain knowledge of psycho-acoustics and speech production to improve coding efficiency. More recently, machine learning models have been successfully applied to the field of audio compression, demonstrating the additional value brought by data-driven solutions. For example, it is possible to apply them as a post-processing step to improve the quality of existing codecs. This can be accomplished either via audio superresolution, i.e., extending the frequency bandwidth [2], via audio denoising, i.e., removing lossy coding artifacts [3], or via packet loss concealment [4], [5].

Other solutions adopt models based on machine learning as an integral part of the audio codec architecture. In these areas, recent advances in text-to-speech (TTS) technology proved to be a key ingredient. For example, WaveNet [6], a strong generative model originally applied to generate speech from text, was adopted as a decoder in a neural codec [7], [8]. Other neural audio codecs adopt different model architectures, e.g., WaveRNN in LPCNet [9] and WaveGRU in Lyra [10], all targeting speech at low bitrates.

In this paper we propose *SoundStream*, a novel audio codec that can compress speech, music and general audio more efficiently than previous codecs, as illustrated in Fig. 1. *SoundStream* leverages state-of-the-art solutions in the field of neural audio synthesis, and introduces a new learnable quantization module, to deliver audio at high perceptual quality, while operating at low-to-medium bitrates. Fig. 2 illustrates the high-level model architecture of the codec. A fully convolutional encoder receives as input a time-domain waveform and produces a sequence of embeddings at a lower sampling rate, which are quantized by a residual vector quantizer. A fully convolutional decoder receives the quantized embeddings and reconstructs an approximation of the original waveform. The model is trained end-to-end using both reconstruction and adversarial losses. To this end, one (or more) discriminators are trained jointly, with the goal of distinguishing the decoded audio from the original audio and, as a by-product, to provide a space where a feature-based reconstruction loss can be computed. Both the encoder and the

Manuscript received July 7, 2021; revised October 8, 2021 and November 5, 2021; accepted November 8, 2021. Date of publication November 23, 2021; date of current version January 24, 2022. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Tom Backstrom. *(Corresponding author: Neil Zeghidour.)*

The authors are with Research Google Inc., Mountain View, CA 94043, USA (e-mail: neilz@google.com; aluebs@google.com; ahmedomran@google.com; jks@google.com; mtagliasacchi@google.com).

Digital Object Identifier 10.1109/TASLP.2021.3129994

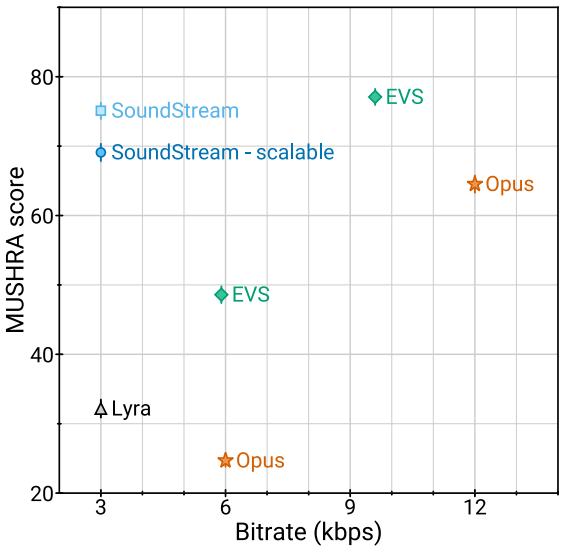


Fig. 1. *SoundStream* @3 kbps vs. state-of-the-art codecs.

decoder only use causal convolutions, so the overall architectural latency of the model is determined solely by the temporal resampling ratio between the original time-domain waveform and the embeddings. In summary, we make the following key contributions:

- We propose *SoundStream*, a neural audio codec in which all the constituent components (encoder, decoder and quantizer) are trained end-to-end with a mix of reconstruction and adversarial losses to achieve superior audio quality.
- We introduce a new residual vector quantizer, and investigate the rate-distortion-complexity trade-offs implied by its design. In addition, we propose a novel "quantizer dropout" technique for training the residual vector quantizer, which enables a single model to handle different bitrates.
- We demonstrate that learning the encoder brings a very significant coding efficiency improvement, with respect to a solution that adopts mel-spectrogram features.
- We demonstrate by means of subjective quality metrics that *SoundStream* outperforms both Opus and EVS over a wide range of bitrates.
- We design our model to support streamable inference, which can operate at low-latency. When deployed on a smartphone, it runs in real-time on a single CPU thread.
- We propose a variant of the *SoundStream* codec that performs joint audio compression and enhancement, without introducing additional latency.

II. Related Work

Traditional audio codecs: Opus [11], EVS [12], and USAC [13] are state-of-the-art audio codecs, which combine traditional coding tools, such as linear predictive techniques and the modified discrete cosine transform, to deliver high coding efficiency over different content types, bitrates and sampling rates, while ensuring low-latency for real-time audio communications. We compare *SoundStream* with both Opus and EVS in our subjective evaluation.

Audio generative models: Several generative models have been developed for converting text or coded features into audio waveforms. WaveNet [6] allows for global and local signal conditioning to synthesize both speech and music. SampleRNN [14] uses recurrent networks in a similar fashion, but it relies on previous samples at different scales. These auto-regressive models deliver very high-quality audio, at the cost of an increased computational complexity, since samples are generated one by one. To overcome this issue, Parallel WaveNet [15] allows for parallel computation, yielding considerable speedup during inference. Other approaches involve lightweight and sparse models [16] and networks mimicking the fast Fourier transform as part of the model [9], [17]. More recently, generative adversarial models have emerged as a solution able to deliver high-quality audio with a lower computational complexity. MelGAN [18] is trained to produce audio waveforms when conditioned on mel-spectrograms, training a multi-scale waveform discriminator together with the generator. HiFiGAN [19] takes a similar approach but it applies discriminators to both multiple scales and multiple periods of the audio samples. The design of the decoder and the losses in *SoundStream* is based on this class of audio generative models.

Audio enhancement: Deep neural networks have been applied to different audio enhancement tasks, ranging from denoising [20]–[24] to dereverberation [25], [26], lossy coding denoising [3] and frequency bandwidth extension [2], [27]. In this paper we show that it is possible to jointly perform audio compression and speech enhancement with a single model, without introducing additional latency.

Vector quantization: Learning the optimal quantizer is a key element to achieve high coding efficiency. Optimal scalar quantization based on Lloyd's algorithm [28] can be extended to a high-dimensional space via the generalized Lloyd algorithm (GLA) [29], which is very similar to k-means clustering [30]. In vector quantization [31], a point in a high-dimensional space is mapped onto a discrete set of code vectors. Vector quantization has been commonly used as a building block of traditional audio codecs [32]. For example, Code-excited Linear Prediction (CELP [33]) encodes an excitation signal via a vector quantizer codebook. More recently, vector quantization has been applied in the context of neural network models to compress the latent representation of input features. For example, in variational autoencoders, vector quantization has been used to generate images [34], [35] and music [36], [37]. Vector quantization can become prohibitively expensive, as the size of the codebook grows exponentially when rate is increased. For this reason, structured vector quantizers [38], [39] (e.g., residual, product, lattice vector quantizers, etc.) have been proposed to obtain a trade-off between computational complexity and coding efficiency in traditional codecs. In *SoundStream*, we extend the learnable vector quantizer of VQ-VAE [34] and introduce a residual (a.k.a. multi-stage) vector quantizer, which is learned end-to-end with the rest of the model. To the best of the authors knowledge, this is the first time that this form of vector quantization is used in the context of neural networks and trained end-to-end with the rest of the model.

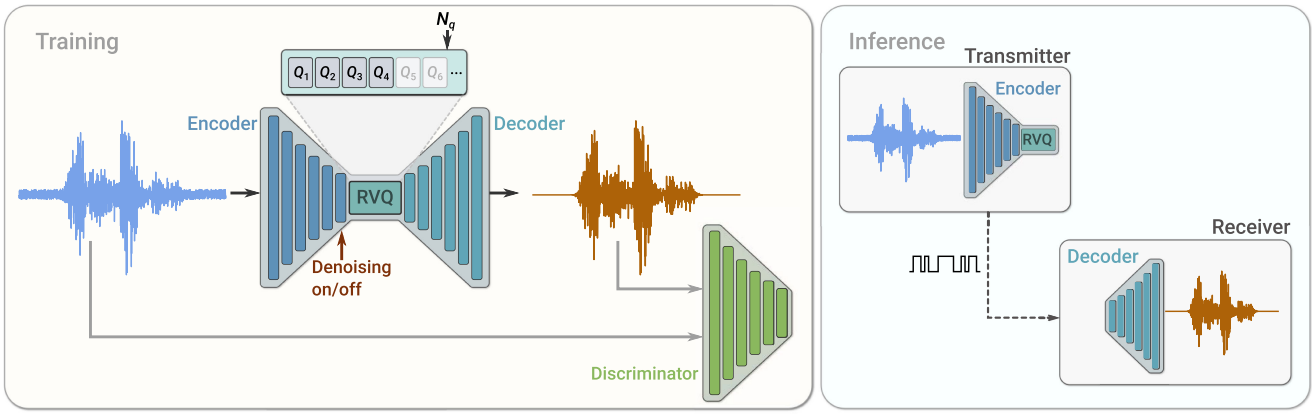


Fig. 2. *SoundStream* model architecture. A convolutional encoder produces a latent representation of the input audio samples, which is quantized using a variable number n_q of residual vector quantizers (RVQ). During training, the model parameters are optimized using a combination of reconstruction and adversarial losses. An optional conditioning input can be used to indicate whether background noise has to be removed from the audio. When deploying the model, the encoder and quantizer on a transmitter client send the compressed bitstream to a receiver client that can then decode the audio signal.

Neural audio codecs: End-to-end neural audio codecs rely on data-driven methods to learn efficient audio representations, instead of relying on handcrafted signal processing components. Autoencoder networks with quantization of hidden features were applied to speech coding early on [40]. More recently, a more sophisticated deep convolutional network for speech compression was described in [41]. Efficient compression of audio using neural networks has been demonstrated in several works, mostly targeting speech coding at low bitrates. A VQ-VAE speech codec was proposed in [8], operating at 1.6 kbps. Lyra [10] is a generative model that encodes quantized mel-spectrogram features of speech, which are decoded with an auto-regressive WaveGRU model to achieve state-of-the-art results at 3 kbps. A very low-bitrate codec proposed in [42] decodes speech representations obtained via self-supervised learning. An end-to-end audio codec targeting general audio at high bitrates (i.e., above 64 kbps) was proposed in [43]. The model architecture adopts a residual coding pipeline, which consists of multiple autoencoding modules and a psycho-acoustic model is used to drive the loss function during training.

Unlike [42] which specifically targets speech by combining speaker, phonetic and pitch embeddings, *SoundStream* does not make assumptions on the nature of the signal it encodes, and thus works for diverse audio content types. While [10] learns a decoder on fixed features, *SoundStream* is trained in an end-to-end fashion. Our experiments (see Section IV) show that learning the encoder increases the audio quality substantially. *SoundStream* achieves bitrate scalability, i.e., the ability of a single model to operate at different bitrates at no additional cost, thanks to its residual vector quantizer and to our original quantizer dropout training scheme (see Section III-C). This is unlike the work in [41], [43], [44] which enforce a specific bitrate and require training a different model for each target bitrate. A single *SoundStream* model is able to compress speech, music and general audio, while operating at a 24 kHz sampling rate and low-to-medium bitrates (3 kbps to 18 kbps in our experiments), in real time on a smartphone CPU. This is the first time that a neural audio codec is shown to outperform state-of-the-art codecs like Opus and EVS over this broad range of bitrates.

Joint compression and enhancement: Recent work has explored joint compression and enhancement. The work in [45] trains a speech enhancement system with a quantized bottleneck. Instead, *SoundStream* integrates a time-dependent conditioning layer, which allows for real-time controllable denoising. As we design *SoundStream* as a general-purpose audio codec, controlling when to denoise allows for encoding acoustic scenes and natural sounds that would be otherwise removed.

III. Model

We consider a single channel recording $x \in \mathbb{R}^T$ of duration T and sampled at f_s. The *SoundStream* model consists of a sequence of three building blocks, as illustrated in Fig. 2:

- an encoder, which maps x to a sequence of embeddings (see Section III-A),
- a residual vector quantizer, which replaces each embedding by the sum of vectors from a set of finite codebooks, thus compressing the representation with a target number of bits (see Section III-C),
- a decoder, which produces a lossy reconstruction $\hat{x} \in \mathbb{R}^T$ from quantized embeddings (see Section III-B).

The model is trained end-to-end together with a discriminator (see Section III-D), using the mix of adversarial and reconstruction losses described in Section III-E. Optionally, a conditioning signal can be added, which determines whether denoising is applied at the encoder or decoder side, as detailed in Section III-F.

A. Encoder Architecture

The encoder architecture is illustrated in Fig. 3 and follows the same structure as the *streaming SEANet* encoder described in [2], but without skip connections. It consists of a 1D convolution layer (with C_{enc} channels), followed by B_{enc} convolution blocks. Each of the blocks consists of three residual units, containing

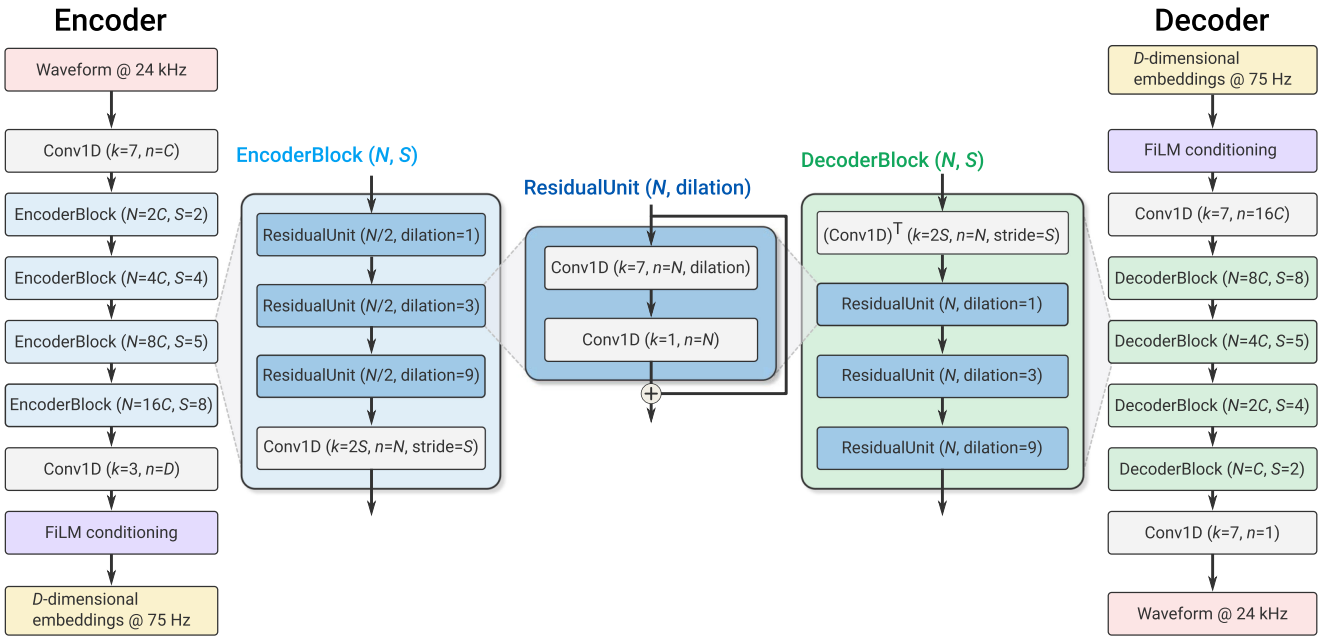


Fig. 3. Encoder and decoder model architecture.

dilated convolutions with dilation rates of 1, 3, and 9, respectively, followed by a down-sampling layer in the form of a strided convolution. The number of channels is doubled whenever down-sampling, starting from C_{enc}. A final 1D convolution layer with a kernel of length 3 and a stride of 1 is used to set the dimensionality of the embeddings to D ($D = 256$ in our experiments). To guarantee real-time inference, all convolutions are *causal*. This means that padding is only applied to the past but not the future in both training and offline inference, whereas no padding is used in streaming inference. We use the ELU activation [46] and we do not apply any normalization. The number B_{enc} of convolution blocks and the corresponding striding sequence determines the temporal resampling ratio between the input waveform and the embeddings. For example, when $B_{\text{enc}} = 4$ and using $(2, 4, 5, 8)$ as strides, one embedding is computed every $M = 2 \cdot 4 \cdot 5 \cdot 8 = 320$ input samples. Thus, the encoder outputs $\text{enc}(x) \in \mathbb{R}^{S \times D}$, with $S = T/M$.

B. Decoder Architecture

The decoder architecture follows a similar design, as illustrated in Fig. 3. A 1D convolution layer is followed by a sequence of B_{dec} convolution blocks. The decoder block mirrors the encoder block, and consists of a transposed convolution for up-sampling followed by the same three residual units. We use the same strides as the encoder, but in reverse order, to reconstruct a waveform with the same resolution as the input waveform. The number of channels is halved whenever up-sampling, so that the last decoder block outputs C_{dec} channels. A final 1D convolution layer with one filter, a kernel of size 7 and stride 1 projects the embeddings back to the waveform domain to produce $\hat{x}$. In Fig. 3, the same number of channels in both the encoder and the decoder is controlled by the same parameter, i.e., $C_{\text{enc}} = C_{\text{dec}} = C$. We also investigate cases in which $C_{\text{enc}} \neq C_{\text{dec}}$, which results in a computationally lighter encoder and a heavier decoder, or vice-versa (see Section V-D).

Algorithm 1: Residual Vector Quantization.

Input: $y = \text{enc}(x)$ the output of the encoder, vector quantizers Q_i for $i = 1..N_q$
Output: the quantized $\hat{y}$
$\hat{y} \leftarrow 0.0$ residual $\leftarrow y$
for $i = 1$ to N_q **do**
  $\hat{y}$ += $Q_i(\text{residual})$
  residual −= $Q_i(\text{residual})$
return $\hat{y}$

C. Residual Vector Quantizer

The goal of the quantizer is to compress the output of the encoder $\text{enc}(x)$ to a target bitrate R, expressed in bits/second (bps). In order to train *SoundStream* in an end-to-end fashion, the quantizer needs to be jointly trained with the encoder and the decoder by backpropagation. The vector quantizer (VQ) proposed in [34], [35] in the context of VQ-VAEs meets this requirement. This vector quantizer learns a codebook of N vectors to encode each D-dimensional frame of $\text{enc}(x)$. The encoded audio $\text{enc}(x) \in \mathbb{R}^{S \times D}$ is then mapped to a sequence of one-hot vectors of shape $S \times N$, which can be represented using $S \log_2 N$ bits.

Limitations of Vector Quantization: As a concrete example, let us consider a codec targeting a bitrate $R = 6000$ bps. When using a striding factor $M = 320$, each second of audio at a sampling rate $f_s = 24000$ Hz is represented by $S = 75$ frames at the output of the encoder. This corresponds to $r = 6000/75 = 80$ bits allocated to each frame. Using a plain vector quantizer, this requires storing a codebook with $N = 2^{80}$ vectors, which is obviously unfeasible.

Residual Vector Quantizer: To address this issue we adopt a Residual Vector Quantizer (a.k.a. multi-stage vector quantizer [39]), which cascades N_q layers of VQ as follows. The unquantized input vector is passed through a first VQ and quantization residuals are computed. The residuals are then iteratively quantized by a sequence of additional $N_q - 1$ vector quantizers, as described in Algorithm 1. The total rate budget is uniformly allocated to each VQ, i.e., $r_i = r/N_q = \log_2 N$. For example, when using $N_q = 8$, each quantizer uses a codebook of size $N = 2^{r/N_q} = 2^{80/8} = 1024$. For a target rate budget r, the parameter N_q controls the trade-off between computational complexity and coding efficiency, which we investigate in Section V-D.

The codebook of each quantizer is trained with exponential moving average updates, following the method proposed in VQ-VAE-2 [35]. We also experimented with the original VQ-VAE layer [34], as well as one using a Gumbel softmax [47] but they performed significantly worse. To improve the usage of the codebooks we use two additional methods. First, instead of using a random initialization for the codebook vectors, we run the k-means algorithm on the first training batch and use the learned centroids as initialization. This allows the codebook to be close to the distribution of its inputs at initialization. Second, as proposed in [37], when a codebook vector has not been assigned any input frame for several batches, we replace it with an input frame randomly sampled within the current batch. More precisely, we track the exponential moving average of the assignments to each vector (with a decay factor of 0.99) and replace the vectors of which this statistic falls below 2.

Enabling bitrate scalability with quantizer dropout: Residual vector quantization provides a convenient framework for controlling the bitrate. For a fixed size N of each codebook, the number of VQ layers N_q determines the bitrate. Since the vector quantizers are trained jointly with the encoder/decoder, in principle a different *SoundStream* model should be trained for each target bitrate. Instead, having a single *bitrate scalable* model that can operate at several target bitrates is much more practical, since this reduces the memory footprint needed to store model parameters both at the encoder and decoder side.

To train such a model, we modify Algorithm 1 in the following way: for each input example, we sample n_q uniformly at random in $[1; N_q]$ and only use quantizers Q_i for $i = 1 \dots n_q$. This can be seen as a form of structured dropout [48] applied to quantization layers. Consequently, the model is trained to encode and decode audio for all target bitrates corresponding to the range $n_q = 1 \dots N_q$. During inference, the value of n_q is selected based on the desired bitrate. Previous models for neural compression have relied on product quantization (e.g. wav2vec 2.0 [49]), or on concatenating the output of several VQ layers [7], [8]. With such approaches, changing the bitrate requires either changing the architecture of the encoder and/or the decoder, as the dimensionality changes, or retraining an appropriate codebook. A key advantage of our residual vector quantizer is that the dimensionality of the embeddings does not change with the bitrate. Indeed, the additive composition of the outputs of each VQ layer progressively refines the quantized embeddings, while keeping the same shape. Hence, no architectural changes are needed in neither the encoder nor the decoder to accommodate different bitrates. In Section V-C, we show that this method allows one to train a single *SoundStream* model, which matches the performance of models trained specifically for a given bitrate.

D. Discriminator Architecture

To compute the adversarial losses described in Section III-E, we define two different discriminators: i) a wave-based discriminator, which receives as input a single waveform; ii) an STFT-based discriminator, which receives as input the complex-valued STFT of the input waveform, expressed in terms of real and imaginary parts. Since both discriminators are fully convolutional, the number of logits in the output is proportional to the length of the input audio. Consistently with previous work [19], we observed during development that a wave-based discriminator was enough to reconstruct speech with high quality, however using both the wave-based and STFT-based discriminators reduces artifacts when compressing music.

For the wave-based discriminator, we use the same multi-resolution convolutional discriminator proposed in [18] and adopted in [50]. Three structurally identical models are applied to the input audio at different resolutions: original, 2-times down-sampled, and 4-times down-sampled. Each single-scale discriminator consists of an initial plain convolution followed by four grouped convolutions, each of which has a group size of 4, a down-sampling factor of 4, and a channel multiplier of 4 up to a maximum of 1024 output channels. They are followed by two more plain convolution layers to produce the final output, i.e., the logits.

The STFT-based discriminator is illustrated in Fig. 4 and operates on a single scale, computing the STFT with a window length of $W = 1024$ samples and a hop length of $H = 256$ samples. A 2D-convolution (with kernel size 7×7 and 32 channels) is followed by a sequence of residual blocks. Each block starts with a 3×3 convolution, followed by a 3×4 or a 4×4 convolution, with strides equal to $(1, 2)$ or $(2, 2)$, where (s_t, s_f) indicates the down-sampling factor along the time and frequency axes. We alternate between $(1, 2)$ and $(2, 2)$ strides, for a total of 6 residual blocks. The number of channels is progressively increased with the depth of the network. At the output of the last residual block, the activations have shape $T/(H \cdot 2^3) \times F/2^6$, where T is the number of samples in the time domain and $F = W/2$ is the number of frequency bins. The last layer aggregates the logits across the (down-sampled) frequency bins with a fully connected layer (implemented as a $1 \times F/2^6$ convolution), to obtain a 1-dimensional signal in the (down-sampled) time domain.

E. Training Objective

Let $\mathcal{G}(x) = \text{dec}(Q(\text{enc}(x))$ denote the *SoundStream* generator, which processes the input waveform x through the encoder, the quantizer and the decoder, and $\hat{x} = \mathcal{G}(x)$ be the decoded waveform. We train *SoundStream* with a mix of losses to achieve both signal reconstruction fidelity and perceptual quality, following the principles of the perception-distortion trade-off discussed in [51].

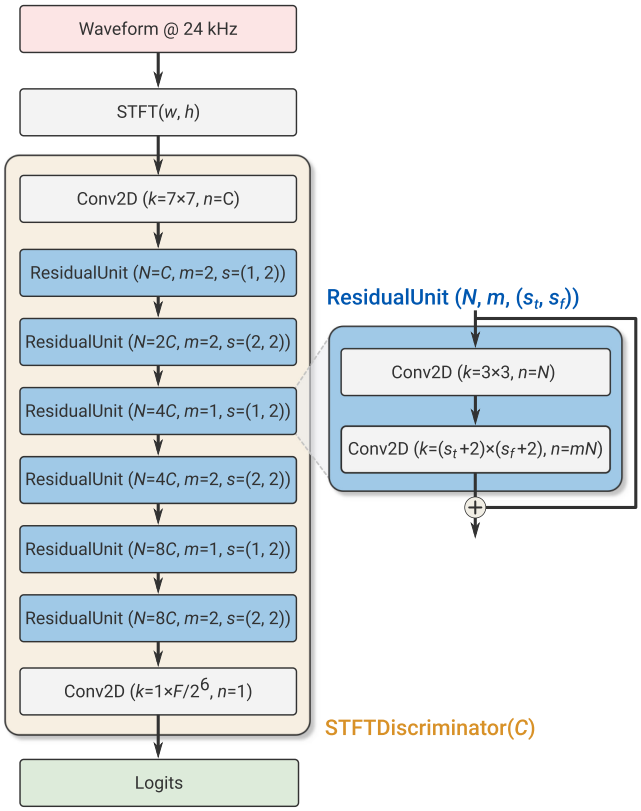


Fig. 4. STFT-based discriminator architecture.

The adversarial loss is used to promote perceptual quality and it is defined as a hinge loss over the logits of the discriminator, averaged over multiple discriminators and over time. More formally, let $k \in \{0, \dots, K\}$ index over the individual discriminators, where $k = 0$ denotes the STFT-based discriminator and $k \in \{1, \dots, K\}$ the different resolutions of the waveform-based discriminator ($K = 3$ in our case). Let T_k denote the number of logits at the output of the k-th discriminator along the time dimension. The discriminator is trained to classify original vs. decoded audio, by minimizing

$$\mathcal{L}_{\mathcal{D}} = E_x \left[\frac{1}{K} \sum_k \frac{1}{T_k} \sum_t \max\left(0, 1 - \mathcal{D}_{k,t}(x)\right) \right] + E_x \left[\frac{1}{K} \sum_k \frac{1}{T_k} \sum_t \max\left(0, 1 + \mathcal{D}_{k,t}(\mathcal{G}(x))\right) \right], \quad (1)$$

while the adversarial loss for the generator is

$$\mathcal{L}_{\mathcal{G}}^{\text{adv}} = E_x \left[\frac{1}{K} \sum_{k,t} \frac{1}{T_k} \max\left(0, 1 - \mathcal{D}_{k,t}(\mathcal{G}(x))\right) \right]. \quad (2)$$

To promote fidelity of the decoded signal $\hat{x}$ with respect to the original x we adopt two additional losses: i) a "feature" loss $\mathcal{L}_{\mathcal{G}}^{\text{feat}}$, computed in the feature space defined by the discriminator(s) [18]; ii) a multi-scale spectral reconstruction loss $\mathcal{L}_{\mathcal{G}}^{\text{rec}}$ [52].

More specifically, the feature loss is computed by taking the average absolute difference between the discriminator's internal layer outputs for the generated audio and those for the corresponding target audio.

$$\mathcal{L}_{\mathcal{G}}^{\text{feat}} = E_x \left[\frac{1}{KL} \sum_{k,l} \frac{1}{T_{k,l}} \sum_t \left| \mathcal{D}_{k,t}^{(l)}(x) - \mathcal{D}_{k,t}^{(l)}(\mathcal{G}(x)) \right| \right], \quad (3)$$

where L is the number of internal layers, $\mathcal{D}_{k,t}^{(l)}$ ($l \in \{1, \dots, L\}$) is the t-th output of layer l of discriminator k, and $T_{k,l}$ denotes the length of the layer in the time dimension.

The multi-scale spectral reconstruction loss follows the specifications described in [53]:

$$\mathcal{L}_{\mathcal{G}}^{\text{rec}} = \sum_{s \in 2^6, \dots, 2^{11}} \sum_t \|\mathcal{S}_t^s(x) - \mathcal{S}_t^s(\mathcal{G}(x))\|_1 + \quad (4)$$

$$\alpha_s \sum_t \| \log \mathcal{S}_t^s(x) - \log \mathcal{S}_t^s(\mathcal{G}(x))\|_2, \quad (5)$$

where $\mathcal{S}_t^s(x)$ denotes the t-th frame of a 64-bin mel-spectrogram computed with window length equal to s and hop length equal to $s/4$. We set $\alpha_s = \sqrt{s/2}$ as in [53].

The overall generator loss is a weighted sum of the different loss components:

$$\mathcal{L}_G = \lambda_{\text{adv}} \mathcal{L}_G^{\text{adv}} + \lambda_{\text{feat}} \cdot \mathcal{L}_G^{\text{feat}} + \lambda_{\text{rec}} \cdot \mathcal{L}_G^{\text{rec}}. \quad (6)$$

In all our experiments we set $\lambda_{\text{adv}} = 1$, $\lambda_{\text{feat}} = 100$ and $\lambda_{\text{rec}} = 1$.

F. Joint Compression and Enhancement

In traditional audio processing pipelines, compression and enhancement are typically performed by different modules. For example, it is possible to apply an audio enhancement algorithm at the transmitter side, before audio is compressed, or at the receiver side, after audio is decoded. In this setup, each processing step contributes to the end-to-end latency, e.g., due to buffering the input audio to the expected frame length determined by the specific algorithm adopted. Conversely, we design *SoundStream* in such a way that compression and enhancement can be carried out jointly by the same model, without increasing the overall latency.

The nature of the enhancement can be determined by the choice of the training data. As a concrete example, in this paper we show that it is possible to combine compression with background noise suppression. More specifically, we train a model in such a way that one can flexibly enable or disable denoising at inference time, by feeding a conditioning signal that represents the two modes (denoising enabled or disabled). To this end, we prepare the training data to consist of tuples of the form: $(\texttt{inputs}, \texttt{targets}, \texttt{denoise})$. When $\texttt{denoise} = \texttt{false}$, $\texttt{targets} = \texttt{inputs}$; when $\texttt{denoise} = \texttt{true}$, $\texttt{targets}$ contain the clean speech component of the corresponding $\texttt{inputs}$. Hence, the network is trained to reconstruct noisy speech if the conditioning signal is disabled, and to produce a clean version of the noisy input if it is enabled. Note that when $\texttt{inputs}$ consist of clean audio (speech or music), $\texttt{targets} = \texttt{inputs}$ and $\texttt{denoise}$ can be either $\texttt{true}$ or $\texttt{false}$. This is done to prevent *SoundStream* from adversely affecting clean audio when denoising is enabled.

To process the conditioning signal, we use Feature-wise Linear Modulation (FiLM) layers [54] in between residual units, which take network features as inputs and transform them as

$$\widetilde{a}_{n,c} = \gamma_{n,c} a_{n,c} + \beta_{n,c}, \tag{7}$$

where $a_{n,c}$ is the n^{th} activation in the c^{th} channel. The coefficients $\gamma_{n,c}$ and $\beta_{n,c}$ are computed by a linear layer that takes as input a (potentially time-varying) two-dimensional one-hot encoding that determines the denoising mode. This allows one to adjust the level of denoising over time.

In principle, FiLM layers can be used anywhere throughout the encoder and decoder architecture. However, in our preliminary experiments, we found that applying conditioning at the bottleneck either at the encoder or at the decoder side (as illustrated in Fig. 3) was effective and no further improvements were observed by applying FiLM layers at different depths. In Section V-E, we quantify the impact of enabling denoising at either the encoder or decoder side both in terms of audio quality and bitrate.

IV. Evaluation Setup

A. Datasets and Training

We train a single *SoundStream* model on three types of audio content: clean speech, noisy speech and music, all at 24 kHz sampling rate. For clean speech, we use the LibriTTS dataset [55], with the following training splits: *train-clean-100*, *train-clean-360* and *train-other-500*. We discard samples that do not contain frequencies above 8 kHz, that is, those which were upsampled from 16 kHz to 24 kHz. This filter results in 30 k, 102 k and 186 k in each of the three splits. Note that some of these samples, especially in the *train-other-500* split, might contain a mild level of early reverberation. For noisy speech, we synthesize samples by mixing speech from LibriTTS with noise from Freesound [56]. This results in a dataset of 37 k noisy samples 2 to 20 s long, selecting noise segments which do not contain speech and have a CC0 license. We apply peak normalization to randomly selected crops of 3 seconds and adjust the mixing gain of the noise component sampling uniformly in the interval $[-30\,\text{dB}, 0\,\text{dB}]$. For music, we use the MagnaTagATune dataset [57], sampling a random subset of 114 k 5-second samples. In addition, we collect a real-world dataset, which contains two speakers (one male and one female speaker) with spontaneous English speech collected indoors in different rooms, located either in an apartment or in an office environment. Each clip is collected with two microphones, one located close to the mouth of the speaker and one at a varying distance of 2 to 5 meters. We select half of the samples from the near-field microphone and half from the far-field microphone, to evaluate on various reverberation conditions. For a subset of the samples, a loudspeaker positioned in the same room is used to generate background noise.

We train with Adam [58], using a learning rate of 10^{-4} and a batch size of 128, for 10^6 steps. As sequences have variable lengths, we prepare batches by cropping random segments of 360 milliseconds. Each segment is normalized to a peak value of 0.95 and multiplied by a random gain in the interval [0.3, 1.0] to ensure that the model is robust to a wide range of amplitudes. We evaluate our models on disjoint test splits of the datasets above. Unless stated otherwise, objective and subjective metrics are computed on a set of 200 audio clips 2–4 seconds long, with 50 samples from each of the four datasets listed above (i.e., clean speech, noisy speech, music, noisy/reverberant speech).

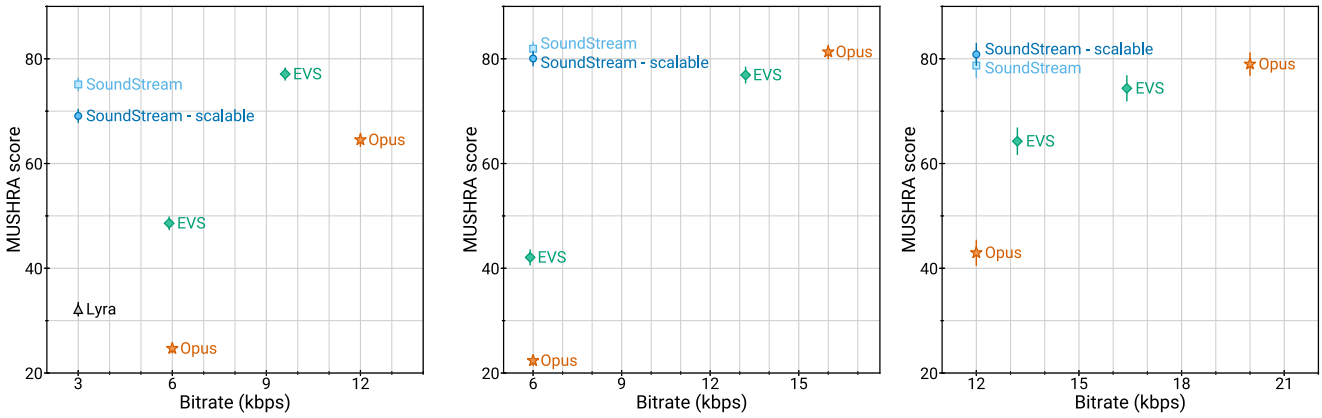
B. Evaluation Metrics

To evaluate *SoundStream*, we perform subjective evaluations by human raters. We have chosen a crowd-sourced methodology inspired by MUSHRA [59], with a hidden reference but no lowpass-filtered anchor. Each of the 200 samples of the evaluation dataset, which include clean, noisy and reverberant speech, as well as music, was rated 20 times. The raters were required to be native English speakers and be using headphones. The number of raters for the 3 kbps, 6 kbps and 12 kbps MUSHRA tests were 275, 273 and 273 respectively. Additionally, to avoid noisy data, a post-screening was put in place to exclude listeners who rated the reference below 90 more than 20% of the time or rated non-reference samples above 90 more than 50% of the time. After post-screening the number of raters was reduced to 131, 76 and 29 respectively. We checked a-posteriori the impact of changing the rejection threshold, but this did not change the outcome of the experiments.

For development and hyperparameter selection, we rely on computational, objective metrics. Numerous metrics have been developed in the past for assessing the perceived similarity between a reference and a processed audio signal. The ITU-T standards PESQ [60] and its replacement POLQA [61] are commonly used metrics. However, both are inconvenient to use owing to licensing restrictions. We choose the freely available and recently open-sourced ViSQOL [62], [63] metric, which has previously shown comparable performance to POLQA. In particular we use "audio" ViSQOL, which operates on audio resampled at 48 kHz. In early experiments, we found this metric to be strongly correlated with subjective evaluations. We thus use it for model selection and ablation studies.

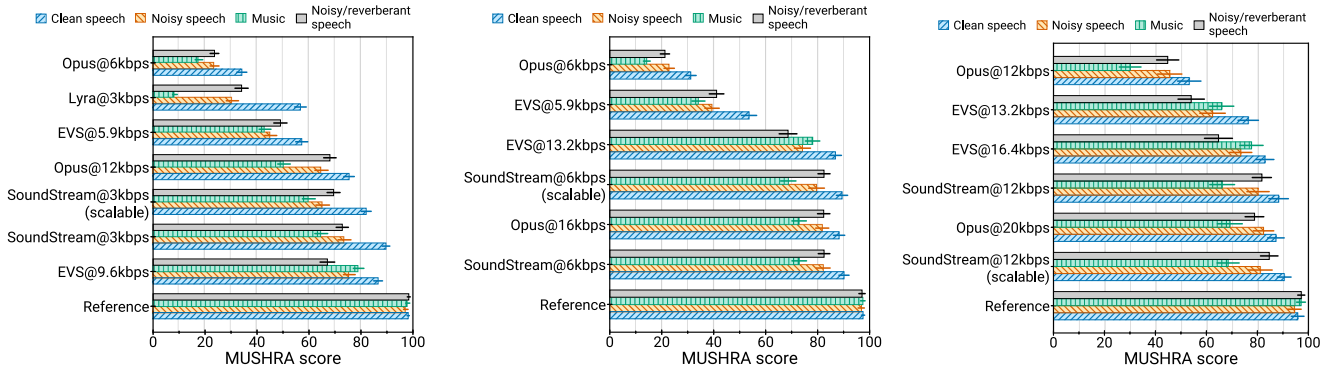
C. Baselines

Opus [11] is a versatile speech and audio codec supporting signal bandwidths from 4 kHz to 24 kHz and bitrates from 6 kbps to 510 kbps. Since its standardization by the IETF in 2012 it has been widely deployed for speech communication over the internet. As the audio codec in applications such as Zoom and applications based on WebRTC [64], [65], such as Microsoft Teams and Google Meet, Opus has hundreds of millions of daily users. Opus is also one of the main audio codecs used in YouTube for streaming. Enhanced Voice Services (EVS) [12] is the latest codec standardized by the 3GPP and was primarily designed for Voice over LTE (VoLTE). Like Opus, it is a versatile codec operating at multiple signal bandwidths, 4 kHz to 20 kHz, and bitrates, 5.9 kbps to 128 kbps. It is replacing AMR-WB [66] and retains full backward operability. We use these two systems as baselines for comparison with the *SoundStream* codec. For the lowest bitrates, we also compare *SoundStream* to the recently proposed Lyra codec [10], an autoregressive generative codec



(a) Low bitrate. (b) Medium bitrate. (c) High bitrate.

Fig. 5. Subjective evaluation results. Error bars denote 95% confidence intervals.



(a) Low bitrate. (b) Medium bitrate. (c) High bitrate.

Fig. 6. Subjective evaluation results by content type. Error bars denote 95% confidence intervals.

operating at 3 kbps. We provide audio processed by *SoundStream* and baselines at different bitrates on a public webpage[1].

V. Results

A. Comparison With Other Codecs

Fig. 5 reports the main result of the paper, where we compare *SoundStream* to Opus and EVS at different bitrates. Namely, we repeated a subjective evaluation based on a MUSHRA-inspired crowdsourced scheme, when *SoundStream* operates at three different bitrates: i) low (3 kbps); ii) medium (6 kbps); iii) high (12 kbps). Fig. 5(a) shows that *SoundStream* at 3 kbps significantly outperforms both Opus at 6 kbps and EVS at 5.9 kbps (i.e., the lowest bitrates at which these codecs can operate), despite using half of the bitrate. To match the quality of *SoundStream*, EVS needs to use at least 9.6 kbps and Opus at least 12 kbps, i.e., 3.2× to 4× more bits than *SoundStream*. We also observe that *SoundStream* outperforms Lyra when they both operate at 3 kbps. We observe similar results when *SoundStream* operates at 6 kbps and 12 kbps. At medium bitrates, EVS and Opus require, respectively, 2.2× to 2.6× more bits to match the same quality. At high bitrates, 1.3× to 1.6× more bits.

Fig. 6 illustrates the results of the subjective evaluation by content type. The quality of *SoundStream* remains consistent when encoding clean speech and noisy speech. In addition, *SoundStream* can encode music when using as little as 3 kbps, with quality significantly better than Opus at 12 kbps and EVS at 5.9 kbps. This is the first time that a codec is shown to operate on diverse content types at such a low bitrate.

B. Objective Quality Metrics

Fig. 7a shows the rate-quality curve of *SoundStream* over a wide range of bitrates, from 3 kbps to 18 kbps. We observe that quality, as measured by means of ViSQOL, gracefully decreases as the bitrate is reduced and it remains above 3.7 even at the lowest bitrate. In our work, *SoundStream* operates at constant bitrate, i.e., the same number of bits is allocated to each encoded frame. At the same time, we measure the bitrate lower bound by computing the empirical entropy of the quantization symbols of the vector quantizers, assuming each vector quantizer to be a discrete memoryless source, i.e., no statistical redundancy is exploited across different layers of the residual vector quantizer, nor across time. Fig. 7a indicates a potential rate saving between 7% and 20%.

[1][Online]. Available: https://google-research.github.io/seanet/soundstream/examples/

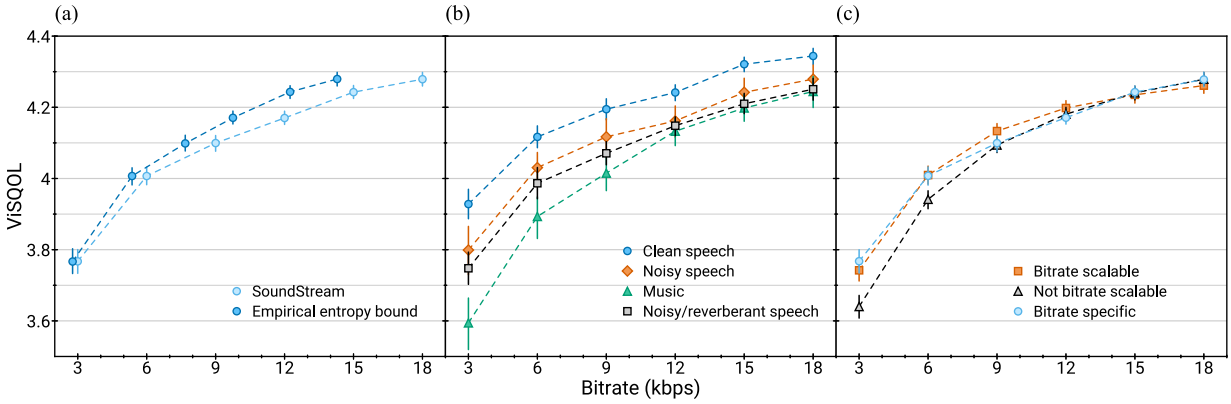


Fig. 7. ViSQOL vs. bitrate. **(a)** *SoundStream* performance on test data, comparing the actual bitrate with the potential bitrate savings achievable by entropy coding **(b)** ViSQOL scores by content type **(c)** Comparison of *SoundStream* models that are trained at 18 kbps with quantizer dropout (bitrate scalable), without quantizer dropout (not bitrate scalable) and evaluated with a variable number of quantizers, or trained and evaluated at a fixed bitrate (bitrate specific). Error bars denote 95% confidence intervals.

We also investigate the rate-quality trade-off achieved when encoding different content types, as illustrated in Fig. 7b. Unsurprisingly, the highest quality is achieved when encoding clean speech. Music represents a more challenging case, due to its inherent diversity of content.

C. Bitrate Scalability

We investigate the bitrate scalability provided by training a single model that can serve different bitrates. To evaluate this aspect, for each bitrate R we consider three *SoundStream* configurations: a) a non-scalable model trained and evaluated at bitrate R (*bitrate specific*); b) a non-scalable model trained at 18 kbps and evaluated at bitrate R by using only the first n_q quantizers during inference (*18 kbps - no dropout*); c) a scalable model trained with quantizer dropout and evaluated at bitrate R (*bitrate scalable*). Fig. 7c shows the ViSQOL scores for these three scenarios. Remarkably, a model trained specifically at 18 kbps retains good performance when evaluated at lower bitrates, even though the model was not trained in these conditions. Unsurprisingly, the quality drop increases as the bitrate decreases, i.e., when there is a more significant difference between training and inference. This gap vanishes when using the quantizer dropout strategy described in Section III-C. Surprisingly, the bitrate scalable model seems to marginally outperform bitrate specific models at 9 kbps and 12 kbps. This suggests that quantizer dropout, beyond providing bitrate scalability, may act as a regularizer.

We confirm these results by including the bitrate scalable variant of *SoundStream* in the MUSHRA subjective evaluation (see Fig. 5). When operating at 3 kbps, the bitrate scalable variant of *SoundStream* is only slightly worse than the bitrate specific variant. Conversely, both at 6 kbps and 12 kbps it matches the same quality as the bitrate specific variant.

D. Ablation Studies

We carried out several additional experiments to evaluate the impact of some of the design choices applied to *SoundStream*. Unless stated otherwise, all these experiments operate at 6 kbps.

TABLE I
AUDIO QUALITY (VISQOL) AND MODEL COMPLEXITY (NUMBER OF PARAMETERS, REAL-TIME FACTOR AND FLOATING-POINT OPERATIONS PER SECOND) FOR DIFFERENT CAPACITY TRADE-OFFS BETWEEN ENCODER AND DECODER, AT 6KBPS

C_{enc}	C_{dec}	#Params	RTF (enc)	RTF (dec)	FLOPS	ViSQOL
32	32	8.4 M	2.4×	2.3×	1.1e10	4.01 ± 0.03
16	16	2.4 M	7.5×	7.1×	3.0e9	3.98 ± 0.03
Smaller encoder						
16	32	5.5 M	7.5×	2.3×	7.0e9	4.02 ± 0.03
8	32	4.8 M	18.6×	2.3×	6.0e9	3.99 ± 0.03
Smaller decoder						
32	16	5.3 M	2.4×	7.1×	7.0e9	3.97 ± 0.03
32	8	4.4 M	2.4×	17.1×	6.0e9	3.90 ± 0.03

Advantage of learning the encoder: We explore the impact of replacing the learnable encoder of *SoundStream* with a fixed mel-filterbank, similarly to Lyra [10]. In this setting, we learn both the quantizer and the decoder and observe a significant drop in objective quality, with ViSQOL going from 3.96 to 3.33. Note that this is significantly worse than what can be achieved when learning the encoder and halving the bitrate (i.e., ViSQOL equal to 3.76 at 3 kbps). This demonstrates that the additional complexity of having a learnable encoder translates to a very significant improvement in the rate-quality trade-off.

Encoder and decoder capacity: The main drawback of using a learnable encoder is the computational cost of the neural architecture, which can be significantly higher than computing fixed, non-learnable features such as mel-filterbanks. For *SoundStream* to be competitive with traditional codecs, not only should it provide a better perceptual quality at an equivalent bitrate, but it must also run in real-time on resource-limited hardware. Table I shows how computational efficiency and audio quality are impacted by the number of channels in the encoder C_{enc} and the decoder C_{dec}. We measured the real-time factor (RTF), defined as the ratio between the temporal length of the input audio and the time needed for encoding/decoding it with *SoundStream*. We profiled these models on a single CPU thread of a Pixel4 smartphone. We observe that the default model

TABLE II
TRADE-OFF BETWEEN RESIDUAL VECTOR QUANTIZER DEPTH AND CODEBOOK SIZE AT 6 KBPS

Number of quantizers N_q	8	16	80
Codebook size N	1024	32	2
ViSQOL	4.01 ± 0.03	3.98 ± 0.03	3.92 ± 0.03

TABLE III
AUDIO QUALITY (VISQOL), REAL-TIME FACTOR (RTF) AND FLOATING-POINT OPERATIONS PER SECOND (FLOPS) FOR DIFFERENT LEVELS OF ARCHITECTURAL LATENCY, DEFINED BY THE TOTAL STRIDING FACTOR OF THE ENCODER/DECODER, AT 6 KBPS

Strides	Latency	N_q	RTF (enc)	RTF (dec)	FLOPS	ViSQOL
$(1, 4, 5, 8)$	7.5ms	4	1.6×	1.5×	2.0e10	4.01 ± 0.02
$(2, 4, 5, 8)$	13ms	8	2.4×	2.3×	1.1e10	4.01 ± 0.03
$(4, 4, 5, 8)$	26ms	16	4.1×	4.0×	6.4e9	4.01 ± 0.03

($C_{\text{enc}} = C_{\text{dec}} = 32$) runs in real-time (RTF $> 2.3\times$). Decreasing the model capacity by setting $C_{\text{enc}} = C_{\text{dec}} = 16$ only marginally affects the reconstruction quality while increasing the real-time factor significantly (RTF $> 7.1\times$). We also investigated configurations with asymmetric model capacities. Using a smaller encoder, it is possible to achieve a significant speedup without sacrificing quality (ViSQOL drops from 3.96 to 3.94, while the encoder RTF increases to $18.6\times$). Instead, decreasing the capacity of the decoder has a more significant impact on quality (ViSQOL drops from 3.96 to 3.84). This is aligned with recent findings in the field of neural image compression [67], which also adopt a lighter encoder and a heavier decoder.

Vector quantizer depth and codebook size: The number of bits used to encode a single frame is equal to $N_q \log_2 N$, where N_q denotes the number of quantizers and N the codebook size. Hence, it is possible to achieve the same target bitrate for different combinations of N_q and N. Table II shows three configurations, all operating at 6 kbps. As expected, using fewer vector quantizers, each with a larger codebook, achieves the highest coding efficiency at the cost of higher computational complexity. Remarkably, using a sequence of 80 1-bit quantizers leads only to a modest quality degradation. This demonstrates that it is possible to successfully train very deep residual vector quantizers without facing optimization issues. On the other side, as discussed in Section III-C, growing the codebook size can quickly lead to unmanageable memory requirements. Thus, the proposed residual vector quantizer offers a practical and effective solution for learning neural codecs operating at high bitrates, as it scales gracefully when using many quantizers, each with a smaller codebook.

Latency: The architectural latency M of the model is defined by the product of the strides, as explained in Section III-A. In our default configuration, $M = 2 \cdot 4 \cdot 5 \cdot 8 = 320$ samples, which means that one frame corresponds to 13.3ms of audio at 24 kHz. The bit budget allocated to the residual vector quantizer needs to be adjusted based on the target architectural latency. For example, when operating at 6 kbps, the residual vector quantizer has a budget of 80 bits per frame. If we double the latency, one frame corresponds to 26.6 ms, so the per-frame budget needs to be increased to 160 bits. Table III compares three configurations, all operating at 6 kbps, where the budget is adjusted by changing the number of quantizers, while keeping the codebook size fixed. We observe that these three configurations are equivalent in terms of audio quality. At the same time, increasing the latency of the model significantly increases the real-time factor, as encoding/decoding of a single frame corresponds to a longer audio sample.

E. Joint Compression and Enhancement

We evaluate a variant of *SoundStream* that is able to jointly perform compression and background noise suppression, which was trained as described in Section III-F. During training, we set the `denoise` flag 50% of the time in each batch. We consider two configurations, in which the conditioning signal is applied to the embeddings: i) one where the conditioning signal is added at the encoder side, just before quantization; ii) another where it is added at the decoder side. For each configuration, we train models at different bitrates. For evaluation we use 1000 samples of noisy speech, generated as described in Section IV-A and compute ViSQOL scores when denoising is enabled or disabled, using clean speech references as targets. Figures 8 shows a substantial improvement of quality when denoising is enabled, with no significant difference between denoising either at the encoder or at the decoder. We observe that the proposed model, which is able to flexibly enable or disable denoising at inference time, does not incur a cost in performance, when compared with a model in which denoising is always enabled. This can be seen comparing Fig. 8c with Fig. 8a and Fig. 8b.

We also investigate whether denoising affects the potential bitrate savings that would be achievable by entropy coding. To evaluate this aspect, we first measured the empirical probability distributions $p_i^{(q)}$, $i = 1 \dots N$, $q = 1 \dots N_q$ on 3200 samples of training data. Then, we measured the empirical distribution $r_i^{(q)}$ on the 1000 test samples and computed the cross-entropy $H(r, p) = -\sum_{i,q} r_i^{(q)} \log_2 p_i^{(q)}$, as an estimate of the bitrate lower bound needed to encode the test samples. Fig. 8 shows that both the encoder-side denoising and fixed denoising offer substantial bitrate savings when compared with decoder-side denoising. Hence, applying denoising before quantization leads to a representation that can be encoded with fewer bits.

F. Joint vs. Disjoint Compression and Enhancement

We compare the proposed model, which is able to perform joint compression and enhancement, with a configuration in which compression is performed by *SoundStream* (with denoising disabled) and enhancement by a dedicated denoising model. For the latter, we adopt SEANet [50], which features a very similar model architecture, with the notable exception of skip connections between encoder and decoder layers and the absence of quantization. We consider two variants: i) one in which compression is followed by denoising (i.e., denoising is applied at the decoder side); ii) another one in which denoising

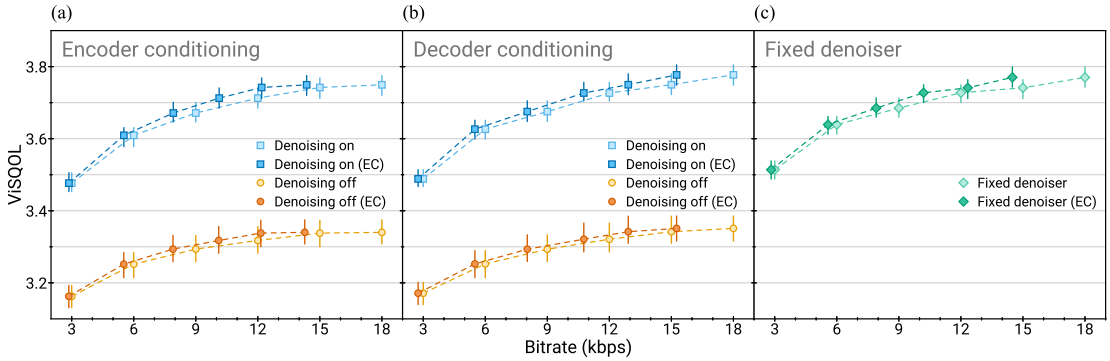


Fig. 8. Performance of *SoundStream* when performing joint compression and background noise suppression, measured by ViSQOL scores at different bitrates. We compare three variants: **(a)** flexible denoising, where the conditioning is added at the encoder side; **(b)** flexible denoising, where the conditioning is added at the decoder side; and **(c)** fixed denoising, where the model was trained to always produce clean outputs. For all models we also report the potential bitrate savings achievable by entropy coding (EC). Error bars denote 95% confidence intervals.

TABLE IV
COMPARISON OF *SOUNDSTREAM* AS A JOINT DENOISER AND CODEC WITH SEANET AS A DENOISER COMPRESSED BY A *SOUNDSTREAM* CODEC ON VCTK SAMPLES AT DIFFERENT SIGNAL-TO-NOISE RATIOS, AS WELL AS "CLEAN" SAMPLES WITH NO ADDED NOISE. UNCERTAINTIES DENOTE 95% CONFIDENCE INTERVALS

	ViSQOL			
Input SNR	*SoundStream* (denoising off)	*SoundStream* (denoising on)	*SoundStream* →SEANet	SEANet→ *SoundStream*
0 dB	2.66 ± 0.02	2.93 ± 0.02	3.08 ± 0.03	3.05 ± 0.02
5 dB	2.96 ± 0.03	3.18 ± 0.02	3.33 ± 0.02	3.32 ± 0.02
10 dB	3.26 ± 0.02	3.42 ± 0.02	3.53 ± 0.02	3.50 ± 0.02
15 dB	3.49 ± 0.02	3.58 ± 0.02	3.65 ± 0.02	3.64 ± 0.02
Clean	3.94 ± 0.01	3.85 ± 0.01	3.84 ± 0.01	3.82 ± 0.01

is followed by compression (i.e., denoising is applied at the encoder side).

We evaluate the different models using the VCTK dataset [68], which was neither used for training *SoundStream* nor SEANet. The input samples are 2 s clips of noisy speech cropped to reduce periods of silence and resampled at 24 kHz. For each of the four input signal-to-noise ratios (0 dB, 5 dB, 10 dB and 15 dB) and clean audio, we run inference on 1000 samples and compute ViSQOL scores. As shown in Table IV, one single model trained for joint compression and enhancement achieves a level of quality that is almost on par with using two disjoint models. Also, the former requires only half of the computational cost and incurs no additional architectural latency, which would be introduced when stacking disjoint models. We also observe that the performance gap decreases as the input SNR increases.

VI. CONCLUSION

We propose *SoundStream*, a novel neural audio codec that outperforms state-of-the-art audio codecs over a wide range of bitrates and content types. *SoundStream* consists of an encoder, a residual vector quantizer and a decoder, which are trained end-to-end using a mix of adversarial and reconstruction losses to achieve superior audio quality. The model supports streamable inference and can run in real-time on a single smartphone CPU. When trained with quantizer dropout, a single *SoundStream* model achieves bitrate scalability with a minimal loss in performance when compared with bitrate-specific models. In addition, we show that it is possible to combine compression and enhancement in a single model without introducing additional latency. In future work, we plan to extend the range of bitrates over which SoundStream operates, covering higher bitrates, aiming to attain perceptually lossless audio and encoding multi-channel audio.

ACKNOWLEDGMENT

The authors would like to thank Yunpeng Li, Dominik Roblek, Félix de Chaumont Quitry and Dick Lyon for their helpful feedback.

REFERENCES

[1] A. S. Spanias, "Speech coding: A tutorial review," *Proc. IEEE*, vol. 82, no. 10, pp. 1541–1582, Oct. 1994.
[2] Y. Li, M. Tagliasacchi, O. Rybakov, V. Ungureanu, and D. Roblek, "Real-time speech frequency bandwidth extension," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2021, pp. 691–695.
[3] A. Biswas and D. Jia, "Audio codec enhancement with generative adversarial networks," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2020, pp. 356–360.
[4] F. Stimberg *et al.*, "WaveNetEQ - packet loss concealment with WaveRNN," in *Proc. 54th Asilomar Conf. Signals, Syst., Comput.*, 2020, pp. 672–676.
[5] S. Pascual, J. Serrà, and J. Pons, "Adversarial auto-encoding for packet loss concealment," 2021, *arXiv:2107.03100*.
[6] A. van den Oord *et al.*, "WaveNet: A generative model for raw audio," 2016, *arXiv:1609.03499*.
[7] W. B. Kleijn *et al.*, "WaveNet based low rate speech coding," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2018, pp. 676–680.
[8] C. Gârbacea *et al.*, "Low bit-rate speech coding with VQ-VAE and a WaveNet decoder," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2019, pp. 735–739.
[9] J.-M. Valin and J. Skoglund, "LPCNet: Improving neural speech synthesis through linear prediction," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2019, pp. 5891–5895.
[10] W. B. Kleijn *et al.*, "Generative speech coding with predictive variance regularization," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2021, pp. 6478–6482.
[11] J.-M. Valin, K. Vos, and T. B. Terriberry, "Definition of the opus audio codec," IETF RFC 6716, 2012, [Online]. Available: https://tools.ietf.org/html/rfc6716
[12] M. Dietz *et al.*, "Overview of the EVS codec architecture," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2015, pp. 5698–5702.

[13] M. Neuendorf *et al.*, "The ISO/MPEG Unified speech and audio coding standard - Consistent high quality for all content types and at all bit rates," *J. Audio Eng. Soc.*, vol. 61, no. 12, pp. 956–977, Dec. 2013.

[14] S. Mehri *et al.*, "SampleRNN: An unconditional end-to-end neural audio generation model," 2017, *arXiv:1612.07837*.

[15] A. van den Oord *et al.*, "Parallel WaveNet: Fast high-fidelity speech synthesis," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 3918–3926.

[16] N. Kalchbrenner *et al.*, "Efficient neural audio synthesis," 2018, *arXiv:1802.08435*.

[17] Z. Jin, A. Finkelstein, G. J. Mysore, and J. Lu, "FFTNet: A real-time speaker-dependent neural vocoder," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2018, pp. 2251–2255.

[18] K. Kumar *et al.*, "MelGAN: Generative adversarial networks for conditional waveform synthesis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019.

[19] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis," 2020, *arXiv:2010.05646*.

[20] X. Feng, Y. Zhang, and J. Glass, "Speech feature denoising and dereverberation via deep autoencoders for noisy reverberant speech recognition," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2014, pp. 1759–1763.

[21] S. Pascual, A. Bonafonte, and J. Serra, "SEGAN: Speech enhancement generative adversarial network," 2017, *arXiv:1703.09452*.

[22] F. G. Germain, Q. Chen, and V. Koltun, "Speech denoising with deep feature losses," 2018, *arXiv:1806.10522*.

[23] D. Rethage, J. Pons, and X. Serra, "A WaveNet for speech denoising," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2018, pp. 5069–5073.

[24] C. Donahue, B. Li, and R. Prabhavalkar, "Exploring speech enhancement with generative adversarial networks for robust speech recognition," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2018, pp. 5024–5028.

[25] T. Ishii, H. Komiyama, T. Shinozaki, Y. Horiuchi, and S. Kuroiwa, "Reverberant speech recognition based on denoising autoencoder," in *Proc. Interspeech*, 2013, pp. 3512–3516.

[26] D. S. Williamson and D. Wang, "Time-frequency masking in the complex domain for speech dereverberation and denoising," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 25, no. 7, pp. 1492–1501, Jul. 2017.

[27] T. Y. Lim, R. A. Yeh, Y. Xu, M. N. Do, and M. Hasegawa-Johnson, "Time-frequency networks for audio super-resolution," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2018, pp. 646–650.

[28] S. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, Mar. 1982.

[29] Y. Linde, A. Buzo, and R. Gray, "An algorithm for vector quantizer design," *IEEE Trans. Commun.*, vol. 28, no. 1, pp. 84–95, Jan. 1980.

[30] J. MacQueen, "Some methods for classification and analysis of multivariate observations," *Proc. 5th Berkeley Symp. Math. Statist. Probability*, 1967, pp. 281–297.

[31] R. Gray, "Vector quantization," *IEEE ASSP Mag.*, vol. 1, no. 2, pp. 4–29, Apr. 1984.

[32] J. Makhoul, S. Roucos, and H. Gish, "Vector quantization in speech coding," *Proc. IEEE*, vol. 73, no. 11, pp. 1551–1588, Nov. 1985.

[33] M. Schroeder and B. Atal, "Code-excited linear prediction (CELP): High-quality speech at very low bit rates," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 1985, pp. 937–940.

[34] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," 2017, *arXiv:1711.00937*.

[35] A. Razavi, A. van den Oord, and O. Vinyals, "Generating diverse high-fidelity images with VQ-VAE-2," 2019, *arXiv:1906.00446*.

[36] S. Dieleman, A. van den Oord, and K. Simonyan, "The challenge of realistic music generation: Modelling raw audio at scale," 2018, *arXiv:1806.10474*.

[37] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, "Jukebox: A generative model for music," 2020, *arXiv:2005.00341*.

[38] B.-H. Juang and A. Gray, "Multiple stage vector quantization for speech coding," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 1982, pp. 597–600.

[39] A. Vasuki and P. Vanathi, "A review of vector quantization techniques," *IEEE Potentials*, vol. 25, no. 4, pp. 39–47, Jul./Aug. 2006.

[40] S. Morishima, H. Harashima, and Y. Katayama, "Speech coding based on a multi-layer neural network," in *Proc. IEEE Int. Conf. Commun., Including Supercomm Tech. Sessions*, 1990, pp. 429–433.

[41] S. Kankanahalli, "End-to-end optimized speech coding with deep neural networks," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2018, pp. 2521–2525.

[42] A. Polyak *et al.*, "Speech resynthesis from discrete disentangled self-supervised representations," 2021, *arXiv:2104.00355*.

[43] K. Zhen, J. Sung, M. S. Lee, S. Beack, and M. Kim, "Cascaded cross-module residual learning towards lightweight end-to-end speech coding," 2019, *arXiv:1906.07769*.

[44] D. Petermann, S. Beack, and M. Kim, "Harp-net: Hyper-autoencoded reconstruction propagation for scalable neural audio coding," 2021, *arXiv:2107.10843*.

[45] J. Casebeer, V. Vale, U. Isik, J.-M. Valin, R. Giri, and A. Krishnaswamy, "Enhancing into the codec: Noise robust speech coding with vector-quantized autoencoders," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2021, pp. 711–715.

[46] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (ELUS)," 2015, *arXiv:1511.07289*.

[47] A. Baevski, S. Schneider, and M. Auli, "vq-wav2vec: Self-supervised learning of discrete speech representations," in *Proc. 8th Int. Conf. Learn. Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=rylwJxrYDS

[48] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[49] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "Wav2vec 2.0: A framework for self-supervised learning of speech representations," 2020, *arXiv:2006.11477*.

[50] M. Tagliasacchi, Y. Li, K. Misiunas, and D. Roblek, "SEANet: A multi-modal speech enhancement network," in *Proc. Interspeech*, 2020, pp. 1126–1130.

[51] Y. Blau and T. Michaeli, "The perception-distortion tradeoff," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 6228–6237.

[52] J. Engel, L. H. Hantrakul, C. Gu, and A. Roberts, "DDSP: Differentiable digital signal processing," 2020, *arXiv:2001.04643*.

[53] A. A. Gritsenko, T. Salimans, R. van denJ. BergSnoek, and N. Kalchbrenner, "A spectral energy distance for parallel speech synthesis," 2020, *arXiv:2008.01160*.

[54] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," *Proc. AAAI Conf. Artif. Intell.*, vol. 32, pp. 3942–3951, 2018.

[55] H. Zen *et al.*, "LibriTTS: A corpus derived from Librispeech for text-to-speech," 2019, *arXiv:1904.02882*.

[56] E. Fonseca *et al.*, "Freesound datasets: A platform for the creation of open audio datasets," in *Proc. 18th Int. Soc. Music Inf. Retrieval Conf.*, 2017, pp. 486–493.

[57] E. Law, K. West, M. I. Mandel, M. Bay, and J. S. Downie, "Evaluation of algorithms using games: The case of music tagging." in *Proc. 10th Int. Soc. Music Inf. Retrieval Conf.*, 2009, pp. 387–392.

[58] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2015, *arXiv:1412.6980*.

[59] ITU-R, "Recommendation B S.1534-1: Method for the subjective assessment of intermediate quality level of coding systems," *Int. Telecommun. Union*, 2001.

[60] ITU, "Perceptual evaluation of speech quality (PESQ): An objective method for end-to-end speech quality assessment of narrow-band telephone networks and speech codecs," *Int. Telecomm. Union*, Geneva, Switzerland, ITU-T Rec. *P.* 862, 2001.

[61] ITU, "Perceptual objective listening quality assessment," *Int. Telecomm. Union*, Geneva, Switzerland, ITU-T Rec. *P.* 863, 2018.

[62] A. Hines, J. Skoglund, A. Kokaram, and N. Harte, "ViSQOL: The virtual speech quality objective listener," in *Proc. Int. Workshop Acoust. Signal Enhancement*, 2012, pp. 1–4.

[63] M. Chinen, F. S. C. Lim, J. Skoglund, N. Gureev, F. O'Gorman, and A. Hines, "ViSQOL v3: An open source production ready objective speech and audio metric," in *Proc. 25th Int. Conf. Qual. Multimedia Experience*, 2020, pp. 1–6.

[64] W3C, "WebRTC 1.0: Real-time communication between browsers," [Online]. Available: 2019, https://www.w3.org/TR/webrtc/

[65] C. Holmberg, S. Håkansson, and G. Eriksson, "Web real-time communication use cases and requirements," IETF RFC 7478, Mar. 2015. [Online]. Available: https://tools.ietf.org/html/rfc7478

[66] B. Bessette *et al.*, "The adaptive multirate wideband speech codec (AMR-WB)," *IEEE Speech Audio Process.*, vol. 10, no. 8, pp. 620–636, Nov. 2002.

[67] F. Mentzer, G. Toderici, M. Tschannen, and E. Agustsson, "High-fidelity generative image compression," 2020, *arXiv:2006.09965*.
[68] J. Yamagishi, C. Veaux, and K. MacDonald, "CSTR VCTK Corpus: English multi-speaker corpus for CSTR voice cloning toolkit (version 0.92)," 2019.

Neil Zeghidour received the Ph.D. degree in machine learning from École Normale Supérieure-PSL, Paris, France, jointly with Facebook AI Research. He is a Senior Research Scientist with Google Research, Brain Team. His main research interest focuses on integrate signal processing with deep learning to design fully learnable classification, processing and synthesis systems of audio.

Alejandro Luebs received the Electronic Engineering degree with focus on signal processing from Instituto Tecnológico Buenos Aires, Buenos Aires, Argentina. He joined Google, to work on audio processing components for real-time communications. He is currently with Lyra Team, working on the next generation of generative speech codecs.

Ahmed Omran received the Ph.D. degree in physics from the University of Munich, Munich, Germany. He was a Postdoc with Harvard University, Cambridge, MA, USA, doing research on experimental quantum simulation platforms. He then joined Google Research, as an AI Resident, where his current research interest focuses on applying representation learning to enable better audio synthesis and compression.

Jan Skoglund received the Ph.D. degree from the Chalmers University of Technology, Gothenburg, Sweden, in 1998. He leads a team with Google, San Francisco, CA, USA, developing speech and audio signal processing components for capture, real-time communication, storage, and rendering. These components were deployed in Google software products, such as meet and hardware products such as Chromebooks. After receiving his Ph.D. degree, he worked on low bit rate speech coding with AT&T Labs-Research, Florham Park, NJ, USA. From 2000 to 2011, he was with Global IP Solutions (GIPS), San Francisco, CA, USA, working on speech and audio processing, such as compression, enhancement, and echo cancellation, tailored for packet-switched networks. GIPS' audio and video technology was found in many deployments by, such as, IBM, Google, Yahoo, WebEx, Skype, and Samsung, and was open-sourced as WebRTC after a 2011 acquisition by Google. Since then, he has been a part of Chrome with Google.

Marco Tagliasacchi (Member, IEEE) received the M.Sc. degree (*cum Laude*) in computer engineering and the Ph.D. degree in information technologies from Politecnico di Milano, Milan, Italy, in 2002 and 2006, respectively. He was a Visiting Scholar with the University of California, Berkeley, CA, USA and a Visiting Academic with Imperial College London, London, U.K. In 2007, he joined as a Faculty Member with the Dipartimento di Elettronica e Informazione, Politecnico di Milano, Italy, focusing on signal processing, particularly its applications in multimedia forensics, coding and communications. He has authored or coauthored several papers in top-rated scientific journals and coordinated two Future and Emerging Technologies projects funded by the European Commission. More recently, he was a Research Scientist with Winton, an investment management fund, contributing to projects focused on using machine learning to forecast asset returns. He is currently with Google Research, working on low-power AI models for audio understanding.